



CNN Introduction

বিস্তারিত বাংলা ML নোট | Intuition · Math · Code · Diagrams

📅 তৈরির তারিখ: ২০২৬-০৪-০৪ | সম্পূর্ণ বাংলায় লেখা ML Notes

১৫. 🖼️ Introduction to CNN & Image Processing

(কেন ANN ছবিতে ব্যর্থ হয়? Pixels, Channels এবং Image Tensors)

সিরিজ: Machine Learning & Deep Learning Notes (বাংলা)

পর্ব: ৩ — Convolutional Neural Networks (CNN)

বিষয়: টপিক ১৫/২৫

তারিখ: ২০২৬-০৪-০৪

১. 🧠 সহজ ভাষায় পরিচিতি (Intuition)

🌟 এই concept কী এবং কেন দরকার?

কল্পনা করো তুমি একটি ফটোগ্রাফের Album দেখছো। তোমার বন্ধু তোমাকে জিজ্ঞেস করল, "এই ছবিতে কি বিড়াল আছে?" তুমি এক নজরেই বলে দিতে পারো — হ্যাঁ বা না।

কিন্তু একটি সাধারণ ANN (Artificial Neural Network) কে যদি বলো "এই ছবিটি দেখে বলো এটি বিড়ালের ছবি কিনা" — ANN একদম কানা হয়ে যাবে! কেন?

🍕 বাস্তুব জীবনের উদাহরণ: পিজ্জার রেসিপি

ধরো তুমি পিজ্জা চেনো। পিজ্জা চেনার মূল বৈশিষ্ট্য হলো:

- গোলাকার আকৃতি
- উপরে cheese এবং topping
- পাতলা বা মোটা base

এখন কেউ যদি পিজ্জার একটি ছবি তুলে সেটিকে **এলোমেলো ছোট ছোট টুকরো** করে তোমার সামনে ছড়িয়ে দেয় এবং বলে "বলো এটি কী?" — তুমি কি বলতে পারবে?

না! কারণ spatial (স্থানিক) সম্পর্ক নষ্ট হয়ে গেছে।

ঠিক এভাবেই সাধারণ ANN ছবি নিয়ে কাজ করে — ছবিকে এলোমেলো list-এ পরিণত করে, তারপর চেনার চেষ্টা করে। ফলাফল? সম্পূর্ণ ব্যর্থতা!

🛒 দোকানের উদাহরণ: মালামাল সাজানো

একটি দোকানের কথা চিন্তা করো যেখানে:

- তাকে তাকে জিনিস **সুশৃঙ্খলভাবে** সাজানো
- তুমি একনজরেই বুঝতে পারো কোন তাকে কী আছে

এখন যদি সব জিনিস এলোমেলো করে একটি বড় বস্তায় ভরে দাও — তুমি কি চোখ বন্ধ করে হাত দিয়ে সঠিক জিনিস খুঁজে পাবে?

ANN ঠিক এই ভুলটাই করে — ছবির সব pixel এলোমেলো করে একটি লম্বা তালিকায় পরিণত করে।

CNN (Convolutional Neural Network) এই সমস্যা সমাধান করে — এটি ছবির spatial structure (স্থানিক কাঠামো) অক্ষুণ্ণ রেখে শেখে।

২. 📖 বিস্তারিত ব্যাখ্যা (Deep Explanation)

২.১ 🖼️ Image কী? (একটি ছবি আসলে কী?)

কম্পিউটারের চোখে একটি ছবি হলো **সংখ্যার একটি বিশাল matrix**।

Grayscale (কালো-সাদা) ছবির ক্ষেত্রে:

প্রতিটি pixel = একটি সংখ্যা (০ থেকে 255)

০ = সম্পূর্ণ কালো

255 = সম্পূর্ণ সাদা

উদাহরণস্বরূপ, একটি ৫×৫ grayscale ছবি দেখতে এরকম:

[0, 50, 100, 150, 200]

[25, 75, 125, 175, 225]

[10, 60, 110, 160, 210]

[30, 80, 130, 180, 230]

[45, 95, 145, 195, 245]

২.২ ●●● Pixel কী?

Pixel শব্দটি এসেছে "Picture Element" থেকে।

একটি ছবির সবচেয়ে ছোট একক হলো Pixel।

একটি ছবি = অনেক অনেক Pixel-এর সমষ্টি

1920×1080 HD ছবিতে = $1920 \times 1080 = 2,073,600$ টি Pixel

প্রতিটি Pixel একটি রঙ ধারণ করে। সেই রঙ তৈরি হয় তিনটি মূল রঙের মিশ্রণে।

২.৩ 🎨 Channels (RGB) কী?

RGB মানে:

- **R** = Red (লাল)
- **G** = Green (সবুজ)
- **B** = Blue (নীল)

প্রকৃতিতে যেকোনো রঙ তৈরি হয় এই তিনটি মূল রঙের বিভিন্ন অনুপাতে মিশিয়ে:

বিশুদ্ধ লাল → R=255, G=0, B=0
 বিশুদ্ধ সবুজ → R=0, G=255, B=0
 বিশুদ্ধ নীল → R=0, G=0, B=255
 হলুদ → R=255, G=255, B=0
 সাদা → R=255, G=255, B=255
 কালো → R=0, G=0, B=0
 কমলা → R=255, G=165, B=0

সুতরাং একটি **Color (RGB) ছবিতে** প্রতিটি pixel-এর জন্য **তিনটি** সংখ্যা দরকার।

একটি Pixel-এর রঙ = (R=200, G=100, B=50)
 ↓
 এটি একটি কমলা রঙ

২.৪ 📦 Image Tensor কী?

Tensor হলো বহুমাত্রিক (multi-dimensional) array বা matrix।

একটি RGB ছবিকে Tensor হিসেবে বলা যায়:

Grayscale ছবি → 2D Tensor (Height × Width)
 RGB ছবি → 3D Tensor (Height × Width × Channels)
 বা (Channels × Height × Width) → PyTorch format

উদাহরণ:

একটি 64×64 pixel-এর রঙিন ছবি:

- Height = 64
- Width = 64
- Channels = 3 (R, G, B)

Tensor Shape = (64, 64, 3) → TensorFlow/Keras format

বা (3, 64, 64) → PyTorch format

মোট সংখ্যা = $64 \times 64 \times 3 = 12,288$ টি মান

২.৫ ❌ কেন ANN ছবিতে ব্যর্থ হয়? (৩টি বড় কারণ)

● কারণ ১: Parameter Explosion (প্যারামিটার বিস্ফোরণ)

ANN-এ ছবি দিতে হলে প্রথমে **Flatten** করতে হয় — মানে 2D/3D কে 1D vector-এ রূপান্তর করতে হয়।

একটি 224×224 রঙিন ছবি:

Input size = $224 \times 224 \times 3 = 150,528$ টি মান

প্রথম Hidden Layer-এ মাত্র 1,000 neuron হলে:

প্যারামিটার = $150,528 \times 1,000 = 150,528,000 \approx 15$ কোটি!

শুধুমাত্র প্রথম layer-এর জন্যই 15 কোটি weight!

সমস্যা:

- Memory অনেক বেশি লাগে
- Training অনেক ধীর হয়
- Overfitting-এর সম্ভাবনা প্রচুর বেড়ে যায়

● কারণ ২: Spatial Structure নষ্ট হয় (স্থানিক সম্পর্ক হারিয়ে যায়)

ছবিকে Flatten করলে pixel-গুলোর মধ্যকার সম্পর্ক নষ্ট হয়ে যায়।

Original Image (3×3):

10	20	30
40	50	60
70	80	90

Flattened (1D):

[10, 20, 30, 40, 50, 60, 70, 80, 90]

↑

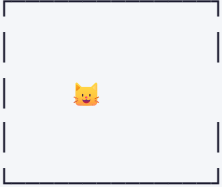
ANN এটাকেই দেখে – কোনো '2D structure' নেই!

- ANN জানে না যে pixel 10 এবং 20 পাশাপাশি আছে
- এটি জানে না যে 10, 40, 70 একটি column তৈরি করে
- প্রান্ত (edge), shape — এসব ধারণা ANN-এর কাছে নেই

● কারণ ৩: Translation Invariance নেই (স্থান পরিবর্তনে অচেনা)

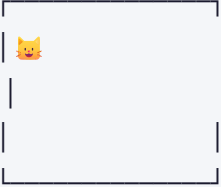
ANN যদি বিড়াল চেনা শেখে ছবির মাঝখানে বিড়াল দেখে, তাহলে সেই বিড়াল যদি ছবির কোণায় চলে যায় — ANN চিনতে পারবে না!

Training ছবি:



ANN শিখেছে ✓

Test ছবি:



ANN চিনতে পারে না! ✗

কারণ pixel index পাল্টে গেছে। ANN "বিড়াল" শেখেনি, শিখেছে "নির্দিষ্ট জায়গায় নির্দিষ্ট সংখ্যার pattern"।

২.৬ ✓ CNN কীভাবে এই সমস্যা সমাধান করে?

সমস্যা	ANN	CNN
Parameter বেশি	✓ হ্যাঁ	✗ না (Weight Sharing)
Spatial structure নষ্ট	✓ হ্যাঁ	✗ না (Convolution)
Translation Invariance নেই	✓ হ্যাঁ	✗ না (Pooling)

CNN-এ:

- **Convolutional Layer** — ছবি scan করে local pattern খোঁজে (edge, texture)
- **Weight Sharing** — একই filter সম্পূর্ণ ছবিতে ব্যবহার, parameter কম
- **Pooling Layer** — ছোট shift/rotation-এ robust থাকে

CNN-এর বিস্তারিত আলোচনা পরের নোটে (টপিক ১৬ ও ১৭) থাকবে।

৩. Math / Theory

৩.১ Image-এর Mathematical Representation

একটি Grayscale ছবি। কে বলা যায়:

$$I \in \mathbb{R}^{(H \times W)}$$

যেখানে:

H = Height (উচ্চতা, pixel সংখ্যা)

W = Width (প্রস্থ, pixel সংখ্যা)

$\mathbb{R}$ = Real numbers (0 থেকে 255 পর্যন্ত)

প্রতিটি element $I(i, j)$ = i -তম row, j -তম column-এর pixel মান

একটি RGB ছবি। কে বলা যায়:

$$I \in \mathbb{R}^{(H \times W \times C)}$$

যেখানে:

H = Height

W = Width

C = Channels (RGB-এর জন্য $C=3$)

$I(i, j, c)$ = i -তম row, j -তম column-এ c -তম channel-এর মান

৩.২ ANN-এ Parameter Calculation (সূত্র)

Fully Connected Layer-এর Parameter সংখ্যা:

$$P = (\text{Input_size} \times \text{Output_neurons}) + \text{Output_neurons}$$

যেখানে:

Input_size = $H \times W \times C$ (Flatten করার পরে)

Output_neurons = Hidden layer-এর neuron সংখ্যা

+Output_neurons = Bias-এর জন্য

Manual Calculation:

উদাহরণ: 32×32 RGB ছবি, প্রথম hidden layer-এ 512 neuron:

Step 1: Input Size = $32 \times 32 \times 3 = 3,072$

Step 2: Weight Parameters = $3,072 \times 512 = 1,572,864$

Step 3: Bias Parameters = 512

Step 4: মোট = $1,572,864 + 512 = 1,573,376 \approx 15.7$ লক্ষ

শুধু প্রথম layer-এই ১৫ লক্ষেরও বেশি parameter!

৩.৩ Pixel Normalization

Machine Learning-এ pixel value সাধারণত normalize করা হয়:

$$\text{Normalized_pixel} = \text{Original_pixel} / 255.0$$

উদাহরণ:

$$\text{Original pixel} = 128$$
$$\text{Normalized} = 128 / 255.0 = 0.502$$
$$\text{Original pixel} = 0 \quad (\text{কালো})$$
$$\text{Normalized} = 0 / 255.0 = 0.0$$
$$\text{Original pixel} = 255 \quad (\text{সাদা})$$
$$\text{Normalized} = 255 / 255.0 = 1.0$$

কেন Normalize করি?

- Neural Network-এর training দ্রুত হয়
- Gradient Explosion/Vanishing কমে
- সব feature একই scale-এ থাকে

৩.৪ RGB থেকে Grayscale রূপান্তর

$$\text{Grayscale} = 0.299 \times R + 0.587 \times G + 0.114 \times B$$

যেখানে coefficients মানুষের চোখের sensitivity অনুযায়ী:

$$0.299 \rightarrow \text{Red sensitivity}$$
$$0.587 \rightarrow \text{Green sensitivity (সবচেয়ে বেশি)}$$
$$0.114 \rightarrow \text{Blue sensitivity (সবচেয়ে কম)}$$

উদাহরণ:

একটি pixel যার R=200, G=150, B=100 হলে:

$$\text{Gray} = 0.299 \times 200 + 0.587 \times 150 + 0.114 \times 100$$
$$= 59.8 + 88.05 + 11.4$$
$$= 159.25 \approx 159$$

8. Code Example (Python)

```

# প্রয়োজনীয় library import করছি
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

# =====
# Part 1: Pixel এবং Image বোঝা
# =====

# হাতে একটি ছোট 3x3 grayscale image তৈরি করছি
grayscale_image = np.array([
    [10, 50, 100], # প্রথম row
    [150, 200, 220], # দ্বিতীয় row
    [80, 30, 255] # তৃতীয় row
], dtype=np.uint8) # uint8 মানে 0-255 range

print("=== Grayscale Image ===")
print(f"Shape: {grayscale_image.shape}") # (3, 3) → 2D Tensor
print(f>Data Type: {grayscale_image.dtype}")
print(f"Image Array:\n{grayscale_image}")
print(f"Pixel at (0,0): {grayscale_image[0, 0]}") # প্রথম pixel মান

# =====
# Part 2: RGB Image তৈরি করা
# =====

# একটি 3x3 RGB image তৈরি করছি (3টি channel)
rgb_image = np.zeros((3, 3, 3), dtype=np.uint8) # (Height, Width, Channels)

# Red channel (প্রথম channel) - সব pixel লাল
rgb_image[:, :, 0] = 200 # R = 200 সব জায়গায়

# Green channel (দ্বিতীয় channel)
rgb_image[:, :, 1] = 100 # G = 100 সব জায়গায়

# Blue channel (তৃতীয় channel)
rgb_image[:, :, 2] = 50 # B = 50 সব জায়গায়

print("\n=== RGB Image ===")
print(f"Shape: {rgb_image.shape}") # (3, 3, 3) → 3D Tensor
print(f"প্রথম pixel (row=0, col=0): {rgb_image[0, 0, :]}") # [R, G, B] মান
print(f"Red Channel:\n{rgb_image[:, :, 0]}")

```

```

print(f"Green Channel:\n{rgb_image[:, :, 1]}")
print(f"Blue Channel:\n{rgb_image[:, :, 2]}")

# =====
# Part 3: Image Tensor Statistics
# =====

# বড় random image তৈরি করি (64x64 RGB)
big_rgb = np.random.randint(0, 256, size=(64, 64, 3), dtype=np.uint8)

print("\n=== 64x64 RGB Image Statistics ===")
print(f"Shape: {big_rgb.shape}") # (64, 64, 3)
print(f"মোট Pixel সংখ্যা: {64 * 64}") # 4096
print(f"মোট Data Point: {big_rgb.size}") # 4096 × 3 = 12288
print(f"সর্বনিম্ন মান: {big_rgb.min()}")
print(f"সর্বোচ্চ মান: {big_rgb.max()}")
print(f"গড় মান: {big_rgb.mean():.2f}")

# =====
# Part 4: Normalization
# =====

print("\n=== Pixel Normalization ===")
original_pixel = 128
normalized = original_pixel / 255.0
print(f"Original pixel value: {original_pixel}")
print(f"Normalized value: {normalized:.4f}")

# পুরো image normalize করা
normalized_image = big_rgb.astype(np.float32) / 255.0
print(f"\nOriginal dtype: {big_rgb.dtype}, Range: [{big_rgb.min()}, {big_rgb.max()}]")
print(f"Normalized dtype: {normalized_image.dtype}, Range: [{normalized_image.min():.2f}, {normalized_image.max():.2f}]")

# =====
# Part 5: ANN-এর Parameter বিস্তারিত প্রদর্শন
# =====

print("\n=== ANN Parameter Explosion Demo ===")

def ann_parameters(height, width, channels, hidden_units):
    """ANN-এর প্রথম layer-এর parameter গণনা"""
    # প্রথমে flatten করতে হবে
    input_size = height * width * channels

```

```

# Weights + Biases
weights = input_size * hidden_units
biases = hidden_units

total = weights + biases
return input_size, total

# বিভিন্ন ছবির size-এর জন্য parameter গণনা
image_sizes = [
    (32, 32, 3, 512, "32x32 ছোট ছবি"),
    (64, 64, 3, 512, "64x64 মাঝারি ছবি"),
    (224, 224, 3, 1000, "224x224 Standard (ImageNet)"),
    (512, 512, 3, 2048, "512x512 বড় ছবি"),
]

print(f"'ছবির Size':<30} {'Input Size':>12} {'Parameters':>20}")
print("-" * 65)
for h, w, c, units, name in image_sizes:
    inp_size, params = ann_parameters(h, w, c, units)
    print(f"{name:<30} {inp_size:>12,} {params:>20,}")

# =====
# Part 6: RGB থেকে Grayscale
# =====

print("\n=== RGB → Grayscale Conversion ===")
r, g, b = 200, 150, 100
gray = 0.299 * r + 0.587 * g + 0.114 * b
print(f"RGB = ({r}, {g}, {b})")
print(f"Grayscale = 0.299×{r} + 0.587×{g} + 0.114×{b}")
print(f"          = {0.299*r:.2f} + {0.587*g:.2f} + {0.114*b:.2f}")
print(f"          = {gray:.2f} ≈ {int(gray)}")

```

 Expected Output:

=== Grayscale Image ===

Shape: (3, 3)

Data Type: uint8

Image Array:

[[10 50 100]

[150 200 220]

[80 30 255]]

Pixel at (0,0): 10

=== RGB Image ===

Shape: (3, 3, 3)

প্রথম pixel (row=0, col=0): [200 100 50]

Red Channel:

[[200 200 200]

[200 200 200]

[200 200 200]]

Green Channel:

[[100 100 100]

[100 100 100]

[100 100 100]]

Blue Channel:

[[50 50 50]

[50 50 50]

[50 50 50]]

=== 64x64 RGB Image Statistics ===

Shape: (64, 64, 3)

মোট Pixel সংখ্যা: 4096

মোট Data Point: 12288

সর্বনিম্ন মান: 0

সর্বোচ্চ মান: 255

গড় মান: 127.xx

=== Pixel Normalization ===

Original pixel value: 128

Normalized value: 0.5020

Original dtype: uint8, Range: [0, 255]

Normalized dtype: float32, Range: [0.00, 1.00]

=== ANN Parameter Explosion Demo ===

ছবির Size

Input Size

Parameters

32×32 ছোট ছবি	3,072	1,573,376
64×64 মাঝারি ছবি	12,288	6,291,456
224×224 Standard (ImageNet)	150,528	150,529,000
512×512 বড় ছবি	786,432	1,610,614,784

=== RGB → Grayscale Conversion ===

RGB = (200, 150, 100)

Grayscale = $0.299 \times 200 + 0.587 \times 150 + 0.114 \times 100$

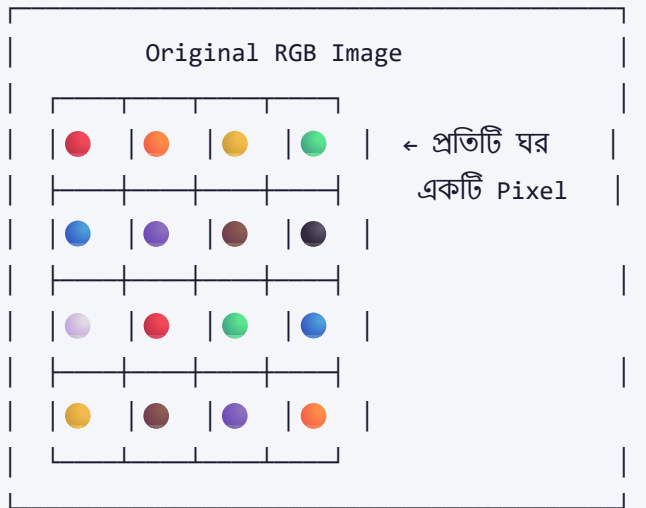
= $59.80 + 88.05 + 11.40$

= $159.25 \approx 159$

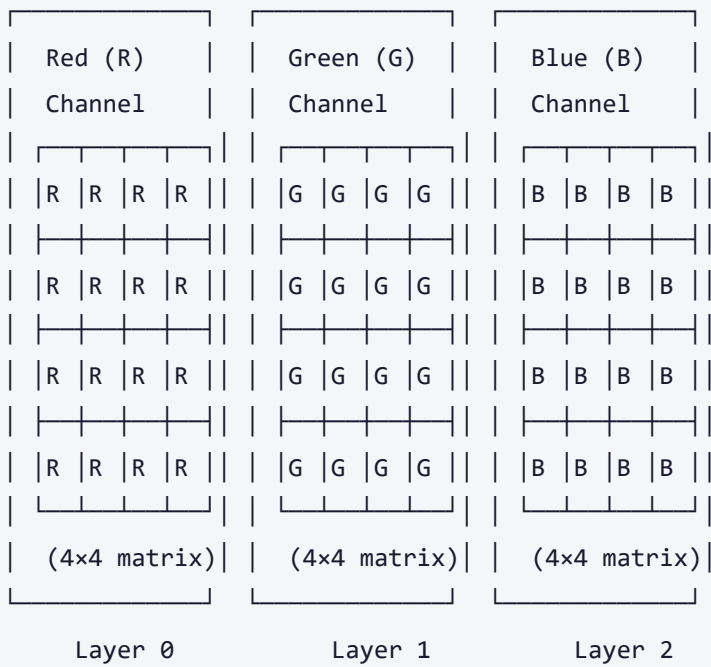
৫. 🎨 Visual / Diagram

৫.১ Pixel এবং RGB Channel-এর চিত্র

একটি রঙিন ছবি (4x4 pixels):



↓ ৩টি আলাদা Channel-এ ভাগ



একত্রিত করলে

Image Tensor Shape = (4, 4, 3)

Height=4, Width=4, Channels=3

৫.২ ANN vs CNN: Flatten-এর সমস্যা

Original 4x4 Image:

P1	P2	P3	P4
P5	P6	P7	P8
P9	P10	P11	P12
P13	P14	P15	P16

ANN-এর জন্য Flatten:

[P1, P2, P3, P4, P5, P6, ...P16]

↑

Sequential list – কোনো spatial info নেই!

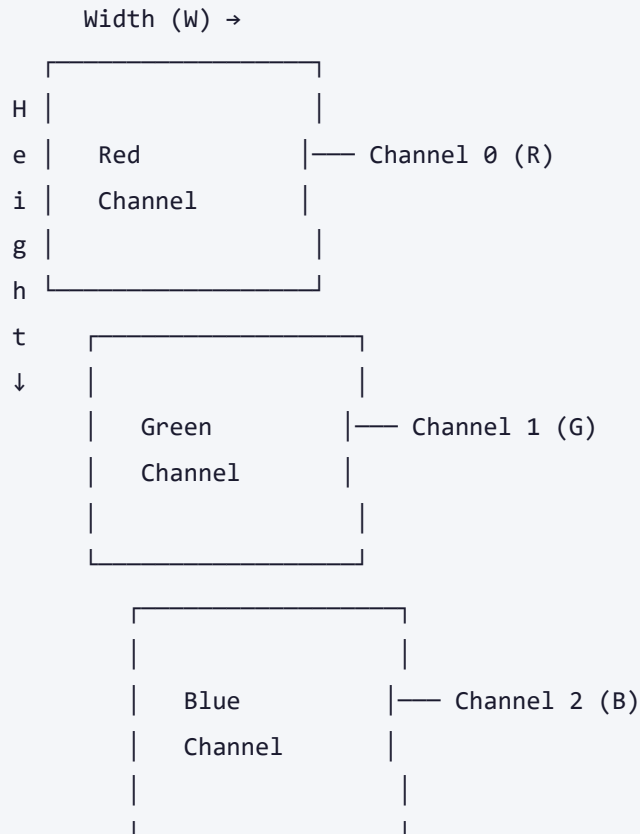
P1 এবং P2 পাশাপাশি → Flatten করলে index 0 এবং 1

P1 এবং P5 একই column → Flatten করলে index 0 এবং 4

ANN সম্পর্ক বোঝে না, CNN বোঝে!

৫.৩ Image Data-এর Tensor Visualization

RGB Image Tensor (3D):



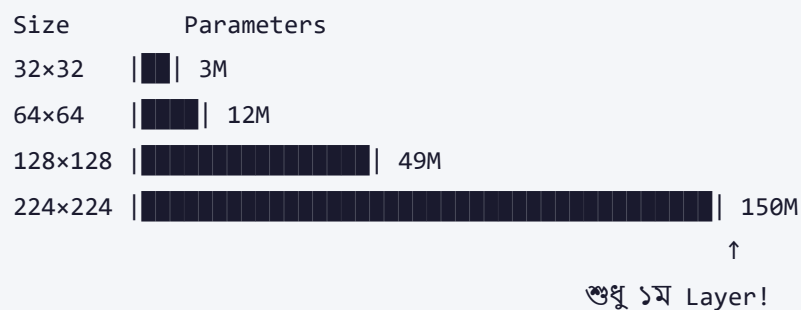
Tensor Shape: (H, W, 3)

যেমন: (224, 224, 3) → TensorFlow/Keras

(3, 224, 224) → PyTorch

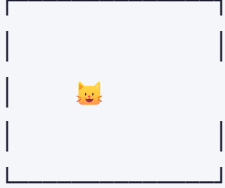
৫.৪ ANN Parameter Explosion Chart

Image Size vs Parameters (Hidden = 1000 neurons):



৫.৬ Translation Invariance সমস্যা

Train Image:



ANN: ✓ Knows it

CNN: ✓ Knows it

Test Image 1:



ANN: ✗ Fails

CNN: ✓ Still works!

Test Image 2:



ANN: ✗ Fails

CNN: ✓ Still works!

৬. ✓ Real-world Use Cases

১. 🏥 Medical Imaging (চিকিৎসা ক্ষেত্রে)

- **Company:** Google Health, IBM Watson
- **Use:** X-ray, MRI scan দেখে রোগ নির্ণয়
- **Example:** CNN দিয়ে ফুসফুসের ক্যান্সার শনাক্ত করা (ANN দিয়ে সম্ভব হতো না)
- প্রতিটি scan = High-resolution image tensor ($512 \times 512 \times 1$ বা বেশি)

২. 🚗 Self-Driving Cars (স্বয়ংচালিত গাড়ি)

- **Company:** Tesla, Waymo
- **Use:** রাস্তার ছবি দেখে object detect করা
- **Example:** পথচারী, ট্র্যাফিক সিগনাল, অন্য গাড়ি সনাক্ত করা
- Camera থেকে আসা প্রতি frame = একটি Image Tensor

৩. 🗠️ Face Recognition (মুখ শনাক্তকরণ)

- **Company:** Apple (Face ID), Facebook
- **Use:** ফোন unlock, ছবিতে মানুষ চেনা
- **Apple Face ID:** infrared image tensor ব্যবহার করে
- ANN দিয়ে এটি করা $10 \times$ বেশি কঠিন ও resource-intensive হতো

8. 🛒 E-commerce Image Search

- **Company:** Amazon, Google Shopping, Pinterest
- **Use:** ছবি দেখে একই রকম product খোঁজা
- **Example:** জামার ছবি উঠিয়ে search করলে অনুরূপ জামা দেখায়

৫. 🌾 Agriculture (কৃষি ক্ষেত্রে)

- **Company:** Microsoft FarmBeats, Blue River Technology
- **Use:** ফসলের রোগ শনাক্ত করা
- Drone থেকে তোলা খামারের ছবি → CNN → রোগাক্রান্ত গাছ চিহ্নিত করা
- ANN দিয়ে এই ছবি process করা প্রায় অসম্ভব ছিল

৭. ⚖️ Pros & Cons

ANN vs CNN: Image Processing-এ তুলনা

বিষয়	ANN (Fully Connected)	CNN
Parameter সংখ্যা	অত্যন্ত বেশি (Explosive)	অনেক কম (Weight Sharing)
Spatial Awareness	নেই	আছে (স্থানিক সম্পর্ক বোঝে)
Translation Invariance	নেই	আছে (Pooling-এর কারণে)
Training Speed	ধীর	দ্রুত
Memory Usage	অনেক বেশি	কম
Overfitting Risk	বেশি	কম
Image Performance	খারাপ	উৎকৃষ্ট
Tabular Data	ভালো	সাধারণত ব্যবহার হয় না

CNN-এর Pros ✅

- ছবির spatial feature নিখুঁতভাবে ধরতে পারে
- Parameter sharing-এর ফলে model ছোট ও দ্রুত
- Translation, rotation-এ robust (টেকসই)
- Hierarchical feature learning (edge → shape → object)
- Transfer Learning সম্ভব (একবার train, বহুবার ব্যবহার)

CNN-এর Cons ❌

- Tabular (ছক আকারের) data-এর জন্য ANN ভালো
- Interpret করা কঠিন ("Black box" সমস্যা)
- Training-এ GPU দরকার
- Large dataset ছাড়া ভালো কাজ করে না
- Architecture design জটিল

৮. ⚠️ Common Mistakes & Gotchas

ভুল ১: Channel Dimension ভুলে যাওয়া

```
# ❌ ভুল: Channel ভুলে গেছে
image = np.zeros((224, 224))      # Grayscale ভাবেছে

# ✅ সঠিক: RGB image
image = np.zeros((224, 224, 3))   # HWC format (Keras)
```

ভুল ২: Channel Order Confusion (TensorFlow vs PyTorch)

```
# TensorFlow / Keras → (Batch, Height, Width, Channels) = NHWC
tf_image = np.zeros((1, 224, 224, 3))

# PyTorch → (Batch, Channels, Height, Width) = NCHW
torch_image = np.zeros((1, 3, 224, 224))

# ❌ ভুল: PyTorch model-এ TensorFlow format দেওয়া
# ✅ সঠিক: প্রতিটি framework-এর format জানা জরুরি
```

ভুল ৩: Normalize না করা

```
# ❌ ভুল: normalize না করে সরাসরি model-এ দেওয়া
image = np.array(image, dtype=np.float32) # 0-255 range

# ✅ সঠিক: 0-1 range-এ normalize করা
image = image / 255.0
```

ভুল ৪: uint8 vs float32 confusion

```
# uint8: মেমরি কম লাগে, কিন্তু calculation-এ ব্যবহার করা যায় না
img_uint8 = np.array([200, 100, 50], dtype=np.uint8)

# float32: calculation-এ ব্যবহার করতে হবে
img_float = img_uint8.astype(np.float32) / 255.0
```

ভুল ৫: ANN দিয়ে Image process করার চেষ্টা

⚠️ সতর্কতা:

Image data → ANN → Flatten করলে spatial information নষ্ট হয়
 Image data → CNN → Spatial information রক্ষা পায়

শুধুমাত্র very small, simple ছবিতে ANN কিছুটা কাজ করতে পারে।
 Real-world image recognition-এ CNN ব্যবহার করো।

ভুল ৬: Channel-এর Range ভুলে বোঝা

Pixel value range: 0 থেকে 255 (uint8)

0.0 থেকে 1.0 (normalized float32)

কোনো pixel-এর value 300 হওয়া অসম্ভব (uint8-এ overflow হয়)!

৯. Related Topics

← আগে যা জানা দরকার (Prerequisites):

1. **ANN Introduction** — Neuron, Layer, Forward Propagation
2. **Backpropagation & Optimizers** — কীভাবে weight update হয়
3. **Activation Functions** — ReLU, Sigmoid-এর intuition
4. **NumPy Basics** — Array, Matrix operation
5. **Linear Algebra** — Matrix multiplication, Dot product

→ পরে কী শেখা উচিত (Next Topics):

1. **The Convolution Operation (টপিক ১৬)** — Kernel/Filter কীভাবে কাজ করে
2. **Pooling Layers (টপিক ১৭)** — Max Pooling, Average Pooling
3. **Famous CNN Architectures** — LeNet, AlexNet, VGG, ResNet
4. **Transfer Learning** — Pre-trained model ব্যবহার করা
5. **Data Augmentation** — Training data বাড়ানোর কৌশল

১০. Memory Tricks

মনে রাখার সহজ কৌশল

Pixel মনে রাখতে:

"**P**icture **E**lement = Pixel" — ছবির সবচেয়ে ছোট কণা

RGB মনে রাখতে:

"*Roktim, Green, Begun* রঙ মিশিয়ে যেকোনো রঙ হয়"

(লাল, সবুজ, বেগুনি = যেকোনো রঙ)

Tensor shape মনে রাখতে:

Keras: "**HWC**" → "Height is first, Width is second, Channels last"

PyTorch: "**CHW**" → "Channels Come first Here with Width"

ANN-এর সমস্যা মনে রাখতে:

"**PST**" rule:

- **P** = Parameter explosion (প্যারামিটার বিস্ফোরণ)

- **S** = Spatial structure নষ্ট

- **T** = Translation invariance নেই

CNN-এর সমাধান মনে রাখতে:

"**LWP**" rule:

- **L** = Local connectivity (local feature শেখা)

- **W** = Weight sharing (একই filter, সব জায়গায়)

- **P** = Pooling (translation robust করে)

✦ ১ লাইনে সারসংক্ষেপ:

"একটি ছবি হলো ($H \times W \times C$) আকারের সংখ্যার Tensor — ANN এই Tensor-কে এলোমেলো list বানিয়ে spatial information নষ্ট করে ও parameter explode করে, তাই CNN ব্যবহার করি যা ছবির structure অক্ষুণ্ন রেখে কাজ করে।"



References

1. CS231n: Convolutional Neural Networks for Visual Recognition — Stanford University
2. Deep Learning by Goodfellow, Bengio, Courville — Chapter 9: Convolutional Networks
3. PyTorch Image Tutorial
4. TensorFlow Image Classification Guide
5. Towards Data Science: Why CNNs?

📅 তৈরির তারিখ: ২০২৬-০৪-০৪ | 📖 পরবর্তী নোট: The Convolution Operation (টপিক ১৬)



Machine Learning & Deep Learning Notes — সম্পূর্ণ বাংলায়



AI-assisted Bengali ML Notes | D:\Coding\Generative-AI\ML_Notes